

Introducción 2º Trimestre

Acceso a Datos + Procesos y Servicios 

Profesor: Amin Harou Azouagh

IES Antonio Gala - 2º DAM

Comenzar →

1. Repaso del 1º Trimestre

Resumen de los dos módulos principales vistos hasta ahora.

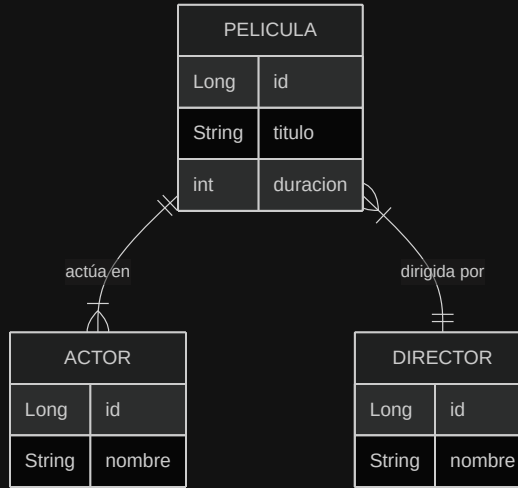
Acceso a Datos (AD)

- Spring Boot: Starter, Estructura, Lombok.
- Capas: Dominio (Entidades) y Repositorio (JPA).
- Relaciones: 1:N y N:M.

Procesos y Servicios (PSP)

- API REST: Controladores y Servicios.
- Concurrencia: Hilos, `ThreadPoolTaskExecutor`.
- Async: Uso de `@Async` para tareas no bloqueantes.

Modelo Entidad-Relación



2. Planificación del Trimestre

July
17

UD	Ra	DUAL	Trimestre
UD 1: Gestión de ficheros y estructuras XML	RA1	0	1 evaluación
UD 3: Persistencia con ORM: introducción a Spring Boot y JPA	RA3	0	
UD 2: Conexión y manipulación de bases de datos relacionales	RA2		
UD 4: Bases de datos objeto-relacionales y orientadas a objetos con Spring Boot	RA4	0	2 evaluación
UD 5: Bases de datos nativas XML	RA5		
UD 6: Componentes de acceso a datos y arquitectura multicapa	RA6	0	3 evaluación

UD	Ra	DUAL	Trimestre
UD 1: Programación multiproceso en Java	RA1	0	1 evaluación
UD 2: Programación multihilo y concurrencia	RA2	0	
UD 3: Spring Boot para concurrencia	RA1 RA2	0	
UD 4: Servicio REST en empresa (CRUD + pruebas y documentación)	RA4		2 evaluación
UD 5: Seguridad básica en servicios REST (auth, roles, validación, evidencias)	RA5		
Aplicaciones y servicios distribuidos con Spring Boot (comunicación en red, sockets, WebSocket y despliegue)	RA3	0	3 evaluación

3. Objetivos de Esta Sesión

Para arrancar el 2º Trimestre, vamos a cerrar dos conceptos clave pendientes:



Mapeo de Datos

Uso profesional de MapStruct para convertir
Entidades ↔ DTOs.



Generación de Datos

Uso de Seeders (DataLoaders) para pruebas
con datos complejos.

Parte 0: Repasando los DTOs

¿Por qué tantos objetos diferentes? 🤔



Tabla Resumen DTOs

Tipo de DTO	Cuándo se usa	Qué representa	Incluye relaciones	Cómo representa relaciones	Quién lo usa	Código HTTP típico
Request / Update DTO	Crear (POST) y Actualizar (PUT)	Intención de creación o cambio	✗ No	IDs (`directorId` , `actorIds`)	Controller → Service	CREATED 201 OK 200 NO CONTENT 204
Response DTO	Respuestas (GET, POST...)	Datos que expone la API	✓ Sí	DTOs simples (sin bucles)	Service → Controller	200 201
Entidad (JPA)	Persistencia	Modelo interno	✓ Sí	Relaciones completas	Repositorio	✗ No se expone



¡OJO! Importante: El DTO es solo una caja de datos.

El DTO NO decide el comportamiento. El Service es quien decide qué hacer con esos datos.



PUT vs PATCH: El Duelo Final 🥊



¿Cuándo se usa PUT?

REEMPLAZO TOTAL 🏗️

Si el recurso no existe, ¡lo crea!



¿Cuándo se usa PATCH?

MODIFICACIÓN PARCIAL 🩹

Solo cambias lo que envías.



Test Rápido

1. "Quiero actualizar solo el email del usuario"



PATCH

2. "Te mando la Película entera nueva para que machaques la vieja"



PUT



Responsabilidad: El Service Manda

El DTO solo transporta datos. El Servicio ejecuta la lógica de negocio.

Controller

Recibe el DTO y se lo pasa al Servicio.

```
service.crearPelicula(peliculaRequestDTO);
```



Service

Decide qué operación ejecutar: Crear, Actualizar o Parchear.

```
// Valida, busca entidades por ID, guarda...
```

Ejemplo de Flujo



Flujo de Datos 1

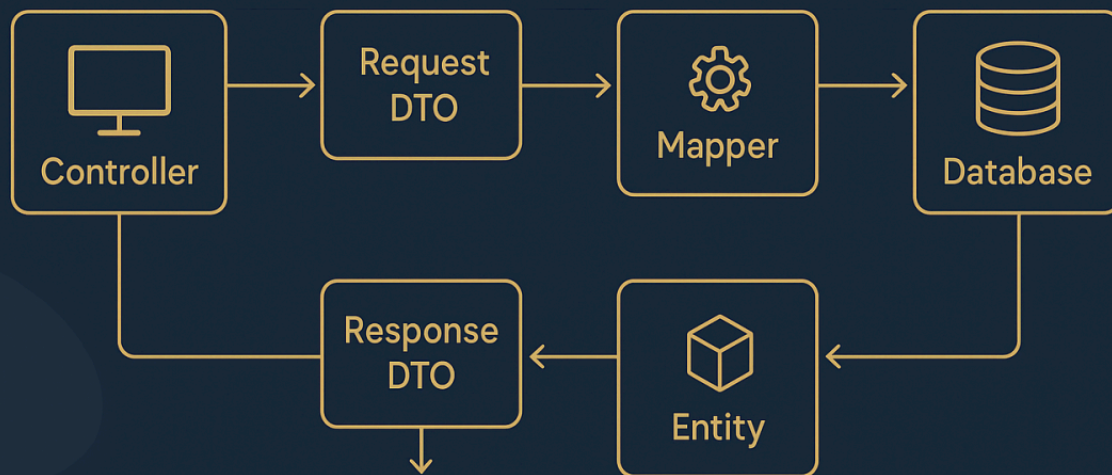




Flujo de Datos 2

DTOs, Entities & Mappers

Keep Them Separate





Las 10 Reglas de Oro

DTOs + MapStruct sin bucles

¿Por qué estas reglas?

✓ Objetivo

- API limpia y mantenible
- Evitar bucles y JSON infinito
- Separar responsabilidades (Controller / Service / Mapper)

✗ Síntomas de mal diseño

- Exponer entidades en endpoints
- DTOs bidireccionales ($A \rightarrow B \rightarrow A$)
- Controllers con lógica y repos dentro

1 Las entidades NO salen por la API

Prohibido

```
1 @GetMapping("/peliculas/{id}")
2 public Pelicula get(@PathVariable Long id) { .
```

Expones tu modelo interno, relaciones y problemas de serialización.

Correcto

```
1 @GetMapping("/peliculas/{id}")
2 public PeliculaResponseDTO get(@PathVariable L
```

La API devuelve DTOs. Las entidades se quedan en backend.

Frase clave: “Entidades para BD, DTOs para la API.”

2 Entrada $\neq$ Salida

DTO Entrada (Request)

Lo que el cliente puede enviar.

- Campos simples
- IDs de relaciones
- Validaciones

DTO Salida (Response)

Lo que tú decides devolver.

- Datos “ricos”
- DTOs anidados (sin bucles)
- Formato pensado para frontend

Regla de oro: Salida = lo que muestro · Entrada = lo que permito cambiar

3 En la ENTRADA: nunca objetos (solo IDs)

Mal (objetos anidados)

```
1 {  
2   "titulo": "Inception",  
3   "director": { "id": 1, "nombre": "Nolan" }  
4 }
```

Riesgo de inconsistencias y creaciones “sin querer”.

Bien (IDs)

```
1 {  
2   "titulo": "Inception",  
3   "directorId": 1,  
4   "actorIds": [2, 3]  
5 }
```

El cliente elige IDs. El backend resuelve relaciones.

4 Las relaciones se resuelven SOLO en el Service 🧠

❌ Mal (controller hace lógica)

```
1 @PostMapping
2 public PeliculaResponseDTO crear(@RequestBody
3     Director d = directorRepo.findById(dto.getDi
4     ...
5 }
```

✅ Bien (service controla)

```
1 public PeliculaResponseDTO crear(PeliculaReque
2     Pelicula p = mapper.toEntity(dto);
3     Director d = directorRepo.findById(dto.getDi
4     p.setDirector(d);
5     return mapper.toResponseDTO(repo.save(p));
6 }
```

Frase clave: “El controller recibe/responde; el service decide.”

5 MapStruct solo traduce (no piensa) 🌐

❌ Mal (lógica en mapper)

```
1 // NO: buscar en BD dentro del mapper
2 Director d = directorRepo.findById(dto.getDire
```

El mapper no debería tener repositorios ni reglas de negocio.

✅ Bien (mapper traduce)

```
1 @Mapper(componentModel = "spring")
2 interface PeliculaMapper {
3     Pelicula toEntity(PeliculaRequestDTO dto);
4     PeliculaResponseDTO toResponseDTO(Pelicula p
5 }
```

Traducción pura: Entity ↔ DTO.

6 En la SALIDA sí hay relaciones, pero cortadas

✓ Correcto

```
1 class PeliculaResponseDTO {
2     DirectorDTO director;
3     List<ActorDTO> actors;
4 }
5
6 class DirectorDTO { Long id; String nombre; }
7 class ActorDTO { Long id; String nombre; }
```

Relaciones visibles pero sin vuelta atrás.

✗ Mal (bucle)

```
1 class ActorDTO {
2     List<PeliculaResponseDTO> peliculas; // bucle
3 }
```

JSON infinito: Película → Actor → Película → ...

7 Nunca DTOs bidireccionales

Regla simple

Si A contiene B en un DTO, entonces B no contiene A.

En entidades JPA puede existir bidireccionalidad; en DTOs, normalmente no.

8 Actualización: los `null` NO machacan (PATCH)

✓ Patrón profesional

```
1 @Mapper(componentModel = "spring")
2 interface PeliculaMapper {
3
4     @BeanMapping(nullValuePropertyMappingStrategy
5     void updateFromDTO(PeliculaUpdateDTO dto,
6                       @MappingTarget Pelicula e
7 }
```

Idea clave

`null` significa: “no quiero cambiar este campo”.

Evita escribir 20 `if(dto.getX() != null)` en el service.

Ejemplo Visual: La Magia del NullStrategy ✨



En Base de Datos

```
id: 1  
titulo: "Avatar"  
director: "Cameron"
```



DTO (Patch)

```
titulo: "Avatar 2"  
director: null
```

(El null significa "no tocar")



Resultado Final

```
id: 1  
titulo: "Avatar 2" ✓  
director: "Cameron" 🛡️ ¡SALVADO!
```



9 Controller: cero lógica, solo HTTP

 Mal

```
1 @PatchMapping("/{id}")
2 public ResponseEntity<?> patch(...) {
3     if(dto.getTitulo()!=null) { ... } // lógica
4     ...
5 }
```

 Bien

```
1 @PatchMapping("/{id}")
2 public ResponseEntity<Void> patch(@PathVariable
3                                   @RequestBody
4                                   service.patch(id, dto);
5     return ResponseEntity.noContent().build(); /
6 }
```

10 Si hay un bucle, el diseño está mal (no se parchea)



✗ “Arreglos” típicos

- `@JsonIgnore` a lo loco
- DTOs enormes con referencias cruzadas
- Devolver entidades “porque es rápido”

✓ Solución real

- Rediseñar DTOs (cortar vuelta atrás)
- Entrada con IDs
- Salida con DTOs simples

Frase final: “Los bucles no se parchean: se diseñan bien.”

Resumen final

Memoriza esto

1. Entidades no salen por API
2. Entrada: IDs · Salida: DTOs
3. Relaciones solo en Service
4. DTOs sin bucles

Resultado

- API estable
- Código limpio
- Menos bugs
- Proyecto “nivel empresa”





Zona de Peligro: ¿Detectas el error?

Observa este código inocente... ¿Qué pasará si pides un ``Actor`` por la API?

```
1 // ActorDTO
2 public class ActorDTO {
3     String nombre;
4     List<PelículaDTO> películas; // <--- 😱
5 }
6
7 // PelículaDTO
8 public class PelículaDTO {
9     String titulo;
10    List<ActorDTO> actores; // <--- 😱
11 }
```

La Pregunta Trampa

Si Antonio Banderas (Actor) sale en Shrek (Película), y Shrek tiene a Antonio Banderas...

¿Qué imprime el navegador?



VER RESPUESTA

Reto Mental #2: El Controlador que sabía demasiado

Analiza este controlador. ¿Qué principio SOLID o arquitectura rompe?

```
1  @PostMapping
2  public PeliculaResponseDTO crear(@RequestBody Pelicula
3      // 🚩 ¿Esto es normal aquí?
4      Director d = directorRepo.findById(dto.getDirectorId())
5          .orElseThrow();
6
7      Pelicula p = new Pelicula();
8      p.setTitulo(dto.getTitulo());
9      p.setDirector(d); // 👉 Asignando relación
10
11      return mapper.toDTO(repo.save(p));
12  }
```

¿Cuál es el veredicto? 

- A) Correcto, el controller orquesta.
- B) Mal, lógica de negocio en controller.
- C) Mal, falta un @Transactional.

Respuesta: B 

El Controller solo debe recibir HTTP y llamar al Service.
Buscar en repositorios y montar relaciones es Lógica de



Reto Mental #3: EL JSON Matrioska

Intentas crear una Película y su Director DE GOLPE:

```
1  /* POST /peliculas */
2  {
3    "titulo": "Titanic 2",
4    "director": {
5      "nombre": "James Cameron", // 👉 Creación imp
6      "nacionalidad": "USA"
7    }
8  }
```

¿Buena idea? 

¿Debemos permitir crear entidades anidadas "al vuelo"?

¡Peligro! 

SÍNTOMA DE MAL DISEÑO

1. Complejidad en validaciones (¿y si el nombre está vacío?).
2. Duplicidad (¿y si Cameron ya existe?).

SOLUCIÓN: Primero crea el Director (recibe ID), luego la Película usando ese ID.



El Hall of Fame de los Errores 🤡



LazyInitException

💀 La Causa:

Intentas leer una lista `@OneToMany` (que es Lazy por defecto) cuando la transacción de base de datos ya se ha cerrado.

```
// Service
@Transactional(readOnly=true) ←
¡Necesario!
public DTO get(Long id) { ... }
```



StackOverflow

💀 La Causa:

Dos objetos se llaman mutuamente en un bucle infinito al imprimirse o serializarse a JSON.

```
@Data // Genera toString() ciclico
class Peli { List<Actor> a; }
class Actor { List<Peli> p; }
```



NullPointerException

💀 La Causa:

Asumes que un dato opcional siempre viene relleno. Mapear a mano es peligroso.

```
// dto.getDirector() es NULL
dto.getDirector().getNombre(); ←
¡CRASH!
```

La Solución Definitiva

Cómo hacerlo bien, paso a paso.

1. El DTO de Entrada (Limpio) 🧹

PeliculaCreateDTO.java

```
1  public class PeliculaCreateDTO {  
2  
3      @NotBlank  
4      private String titulo;  
5  
6      // 🧹 SoloReferencias. Nada de objetos.  
7      @NotNull  
8      private Long directorId;  
9  
10     private List<Long> actorIds;  
11 }
```

👉 Filosofía

- El cliente conoce los IDs.
- El JSON es plano y ligero.
- No hay ambigüedad: "Úsame este director que ya existe".

2. El Service (El Cerebro) 🧠

```
1  @Service
2  public class PeliculaService {
3      // ... inyecciones
4
5      public PeliculaResponseDTO crear(PeliculaCreatedDTO dto) {
6          // 1. Convertir datos simples
7          Pelicula p = mapper.toEntity(dto);
8
9          // 2. Cargar referencias (Lógica pura)
10         Director d = directorRepo.findById(dto.getDirectorId())
11             .orElseThrow(() → new RuntimeException("Director no encontrado"));
12
13         List<Actor> actores = actorRepo.findAllById(dto.getActorIds());
14
15         // 🛡 Validación extra: ¿Están todos?
16         if(actores.size() ≠ dto.getActorIds().size()) throw new RuntimeException("Faltan actores");
17
18         // 3. Establecer relaciones (JPA necesita ENTIDADES, no IDs)
19         p.setDirector(d);
20         p.setActores(actores);
21
22         return mapper.toResponseDTO(repo.save(p));
23     }
24 }
```


3. EL DTO de Salida (Sin Retorno) 🚀

```
1  // PeliculaResponseDTO
2  public class PeliculaResponseDTO {
3      Long id;
4      String titulo;
5      DirectorDTO director; // Objeto completo
6      List<ActorDTO> actores; // Lista de objetos
7  }
8
9  // DirectorDTO (¡Plano!)
10 public class DirectorDTO {
11     Long id;
12     String nombre;
13     // 🚫 List<Peli> peliculas; NO!
14 }
15
16 // ActorDTO (¡Plano!)
17 public class ActorDTO {
18     Long id;
19     String nombre;
20     // 🚫 List<Peli> peliculas; NO!
21 }
```



Resultado

Al pedir la película, ves los detalles del director y actores.

Pero al ver el director... ¡se acabó el camino!

No vuelve a mostrar películas. CERO BUCLES.

Parte 1: MapStruct

Del problema real a la solución profesional

1 El Problema Real: Mapeo Manual 🤔

```
1 // Método manual propenso a errores
2 public PeliculaDTO toDTO(Pelicula p) {
3     if (p == null) return null;
4
5     PeliculaDTO dto = new PeliculaDTO();
6     dto.setId(p.getId());
7     /* Setters infinitos... */
8
9     if (p.getDirector() != null) {
10         dto.setDirectorNombre(p.getDirector().getNombr
11     }
12
13     List<String> actores = new ArrayList<>();
14     for (Actor a : p.getActores()) {
15         actores.add(a.getNombre());
16     }
17     dto.setActores(actores);
18
19     return dto;
20 }
```

✗ Problemas reales

- Código repetitivo (boilerplate).
- Difícil de mantener.
- Fácil de romper si cambia la entidad.

2 ¿Qué es MapStruct?

Idea clave

MapStruct genera código Java en tiempo de compilación.

No usa reflexión, no actúa en ejecución y no es magia.

¿Qué significa eso?

- Lees una interfaz.
- MapStruct genera una clase real.
- El código existe y se puede depurar.

3 ¿Qué ocurre al compilar?

Cuando haces ``mvn compile``:

Interfaz → Implementación → Código Java real

Si no compilas, la clase generada no existe.

4 La Interfaz Mapper (lo que escribimos nosotros)

```
1 @Mapper(componentModel = "spring")
2 public interface DirectorMapper {
3
4     DirectorDTO toDTO(Director director);
5     Director toEntity(DirectorDTO dto);
6 }
```



Importante

- No es una clase.
- No tiene implementación.
- Es una plantilla para generar código.

5 ¿Qué genera MapStruct realmente?

A partir de la interfaz, MapStruct genera una clase:

`DirectorMapperImpl`

Esta clase implementa los métodos y contiene código Java normal (getters, setters, null-checks...).

6 Conexión con Spring

`@Mapper(componentModel = "spring")`

- MapStruct añade `@Component`.
- Spring detecta el bean.
- Se puede inyectar.

Resultado

El mapper se usa como cualquier Service o Repository.

7 Uso desde un Service

```
1  @Service
2  @RequiredArgsConstructor
3  public class DirectorService {
4
5      private final DirectorMapper directorMapper;
6      private final DirectorRepository repo;
7
8      public DirectorDTO findById(Long id) {
9          Director dir = repo.findById(id).orElseThrow();
10         return directorMapper.toDTO(dir);
11     }
12 }
```



Ciclo completo

1. Compilas.
2. Se genera la implementación.
3. Spring la inyecta.

8 Resumen mental

MapStruct no elimina código.

Escribe el código por ti.

Interfaces → Código real → Spring Beans

9 Composición de Mappers (`uses`)

Cuando una Entidad tiene relaciones, podemos delegar el mapeo en otros componentes.

```
1  @Mapper(componentModel = "spring",
2          uses = { DirectorMapper.class, ActorMapper.class })
3  public interface PeliculaMapper {
4
5      // MapStruct detectará automáticamente que debe
6      // DirectorMapper y ActorMapper para los campos
7      // 'director' y la lista 'actors'.
8      PeliculaDTO toDTO(Pelicula pelicula);
9
10     Pelicula toEntity(PeliculaDTO dto);
11 }
```

¿Cómo funciona?

Al añadir `uses`, le decimos a MapStruct:

- "Si no sabes mapear un tipo (ej. `Director`), pregunta a estos mappers".
- MapStruct buscará en `DirectorMapper` un método que convierta de `Director` a `DirectorDTO`.

10 Reglas de Coincidencia

Para que la "magia" funcione, los nombres deben hablar el mismo idioma.

¿Dónde debe coincidir?

El nombre del atributo (field) definido en la clase origen debe ser idéntico al de la clase destino.

```
Entity: String titulo;  
DTO: String titulo;
```

El Desajuste

Si en la Entidad se llama ``name`` y en el DTO ``nombre``, MapStruct no podrá vincularlos automáticamente.

1 1 ¿Y si no coinciden? 🛠️

Cuando los nombres son diferentes o queremos lógica personalizada, usamos `@Mapping`.

```
1 @Mapper(componentModel = "spring")
2 public interface PeliculaMapper {
3
4     // Explicitamos la relación de campos
5     @Mapping(source = "longitud", target = "duracion")
6     @Mapping(source = "director.nombre", target = "nombreDirector")
7     PeliculaDTO toDTO(Pelicula p);
8 }
```

`source`: Atributo en el objeto origen (ej. Entidad).

`target`: Atributo en el objeto destino (ej. DTO).

Parte 2: MapStruct

Ejemplo Guiado - Cómo evitar bucles en Película 🎬 – Director
🎥 – Actor 🤖🧠



Objetivo del ejemplo

- Entender dónde nacen los bucles en una API REST.
- Ver cómo se evitan usando DTOs bien diseñados.
- Diferenciar claramente lectura y escritura de datos.



Requisitos previos (imprescindibles)

Conocimientos previos

- Relaciones JPA: `@ManyToOne`, `@OneToMany`, `@ManyToMany`.
- Capas: Controller / Service / Repository.
- Qué es un DTO y para qué sirve.

Reglas activas

- Las entidades no se exponen.
- En entrada solo IDs.
- Los DTOs no son bidireccionales.



Entidades (donde SÍ hay relaciones)

- 1 Película → Director (N:1)
- 2 Película → Actor (N:M)

✓ Aquí puede haber bidireccionalidad.

✗ Estas clases NO salen por la API.

✗ El problema: bucle infinito

DTO mal diseñado

```
1  class PeliculaDTO {  
2      DirectorDTO director;  
3      List<ActorDTO> actors;  
4  }  
5  
6  class ActorDTO {  
7      List<PeliculaDTO> peliculas; // 🔥 bucle  
8  }
```

Resultado: JSON infinito → error en ejecución.

Qué ocurre

- Pelicula → Actor
- Actor → Pelicula
- Serialización sin fin



Solución: cortar el bucle con DTOs

DTOs correctos

```
1  class PeliculaResponseDTO {
2      Long id;
3      String titulo;
4      DirectorDTO director;
5      List<ActorDTO> actors;
6  }
7
8  class DirectorDTO {
9      Long id;
10     String nombre; // ✗ sin peliculas
11 }
12
13 class ActorDTO {
14     Long id;
15     String nombre; // ✗ sin peliculas
16 }
```

Regla aplicada

Las relaciones se muestran solo en una dirección.



Escritura de datos (POST)

DTO de entrada

```
1  class PeliculaRequestDTO {  
2      String titulo;  
3      Long directorId;  
4      List<Long> actorIds;  
5  }
```



Solo IDs



Nunca objetos completos



Service: dónde se montan las relaciones

```
1  public PeliculaResponseDTO crear(PeliculaRequestDTO dto) {  
2  
3      Pelicula p = mapper.toEntity(dto);  
4  
5      Director d = directorRepo.findById(dto.getDirectorId()).orElseThrow();  
6      List<Actor> actores = actorRepo.findAllById(dto.getActorIds());  
7  
8      p.setDirector(d);  
9      p.setActors(actores);  
10  
11     return mapper.toResponseDTO(peliculaRepo.save(p));  
12 }
```

Idea clave: El mapper no busca en BD. El service sí.



Lectura de datos (GET)

Entidad → Mapper → DTO de salida



Se muestran director y actores



Sin referencias de vuelta



Resumen mental del ejemplo

Escritura

- DTO con IDs
- Service resuelve relaciones
- Se guarda entidad

Lectura

- Entidad completa
- Mapper crea DTO
- Relaciones cortadas

Conclusión: Los bucles no se arreglan con anotaciones, se evitan diseñando bien los DTOs.

Actividad A1 - Cambiar el mapeo de DTOs

Usando Mapper de SpringBoot

Objetivo de la Actividad

Refactorizar el proyecto para usar 2 DTOs por Entidad y separar responsabilidades.

Clases Obligatorias:

-  `Película`
-  `Director`
-  `Actor`

Requisitos Obligatorios

▪ Entrada (Request):

Solo datos primitivos e IDs (nada de objetos anidados).

▪ Salida (Response):

Datos extendidos y relaciones controladas (sin bucles).

▪ Mappers:

Usar `@Mapper(uses = ...)` para reutilizar lógica.

Solución A1: Separación de DTOs

1. Definición de DTOs

```
1 // INPUT: Lo que envían (POST/PUT)
2 public class PeliculaRequestDTO {
3     private String titulo;
4     private Long directorId;    // Solo ID
5     private List<Long> actorIds; // Solo IDs
6 }
7
8 // OUTPUT: Lo que devolvemos (GET)
9 public class PeliculaResponseDTO {
10     private Long id;
11     private String titulo;
12     private DirectorDTO director; // Objeto completo
13     private List<ActorDTO> actores;
14 }
```

2. El Mapper Inteligente

```
1 @Mapper(componentModel = "spring",
2         uses = { DirectorMapper.class, ActorMapper.class })
3 public interface PeliculaMapper {
4
5     // 1. De DTO a Entidad (Ignoramos relaciones con
6     @Mapping(target = "director", ignore = true)
7     @Mapping(target = "actores", ignore = true)
8     Pelicula toEntity(PeliculaRequestDTO dto);
9
10    // 2. De Entidad a Response (MapStruct usa los '
11    PeliculaResponseDTO toResponse(Pelicula entity);
12 }
```

Nota: En `toEntity`, ignoramos director/actores porque los cargaremos manualmente en el Service usando los IDs del DTO.

Parte 2: Seeders (Carga de Datos)

Creando un universo de pruebas 

Teoría: Relaciones y Orden

Al insertar datos, el ORDEN importa para no violar Claves Foráneas.

1. Independientes Primero

No puedes crear una película sin Director (si es obligatorio).

1. Guardar Actores (sin dependencias). 2. Guardar Directores (sin dependencias). 3. Guardar Películas (asignando director y actores).

2. Cuidado con los Bucles

En relaciones bidireccionales (`Many-to-Many`), evita intentar guardar ambos lados a la vez de forma recursiva.

Estrategia: Crea la Entidad Principal (`Pelicula`), añádele las secundarias (`Actores`) y guarda la Principal.

Ejemplo Práctico: Constructor Style

```
1  @Component
2  @RequiredArgsConstructor
3  public class CineDataLoader implements CommandLineRunner {
4
5      // Inyectamos repositorios (Lombok)
6      private final PeliculaRepository peliculaRepo;
7      private final DirectorRepository directorRepo;
8
9      @Override
10     public void run(String... args) {
11         if (peliculaRepo.count() > 0) return;
12
13         // 1. Crear y Guardar Independientes (CONSTRUCTOR)
14         Director nolan = new Director(null, "C. Nolan");
15         // Importante: Guardar para generar ID
16         directorRepo.save(nolan);
17
18         // 2. Crear Dependientes (Pelicula)
19         Pelicula inception = new Pelicula(
20             null, "Inception", 148, nolan, new ArrayList<>());
21     };
22
23     // 3. Añadir Many-to-Many
24     inception.getActors().add(dicaprio);
```

¿Por qué Constructor?

- Si usas `@AllArgsConstructor` de Lombok.
- Más rápido de escribir si tienes pocos campos.
- Nota: El primer argumento suele ser `null` porque el ID lo genera la Base de Datos.



Ejercicio Final: El Relleno Masivo

Misión: Crear un `CineDataLoader`` robusto.

Requisitos:

1. Inserta 5 Directores famosos (Spielberg, Tarantino, Scorsese...).
2. Inserta 10 Actores variados.
3. Inserta 10 Películas reales.
4. Relaciones: Cada película debe tener su director correcto y al menos 2 actores del reparto real.
5. Usa Constructores (``new ...``) en lugar de Builder.



Solución Ejercicio Final

```
1  @Component
2  @RequiredArgsConstructor
3  public class SolucionDataLoader implements CommandLineRunner {
4      private final DirectorRepository directorRepo;
5      private final ActorRepository actorRepo;
6      private final PeliculaRepository peliculaRepo;
7
8      @Override
9      public void run(String... args) {
10         if (directorRepo.count() > 0) return;
11
12         // 1. Directores
13         Director tarantino = directorRepo.save(new Director(null, "Tarantino"));
14         Director scorsese = directorRepo.save(new Director(null, "Scorsese"));
15
16         // 2. Actores
17         Actor deNiro = actorRepo.save(new Actor(null, "Robert De Niro", "USA"));
18         Actor travolta = actorRepo.save(new Actor(null, "John Travolta", "USA"));
19
20         // 3. Peliculas (Helper para limpiar código)
21         crearPelicula("Pulp Fiction", tarantino, Arrays.asList(travolta));
22         crearPelicula("Taxi Driver", scorsese, Arrays.asList(deNiro));
23     }
24 }
```

Resumen de la Actividad A1

¿Qué hemos aprendido?



DTOs

- Entrada: Solo IDs y datos básicos.
- Salida: Objetos completos y relaciones.
- Regla: Nunca exponer Entidades.



MapStruct

- Interfaces: Código generado.
- Uses: Composición de mappers.
- Ignore: Control manual de campos.



Seeders

- Orden: Respetar dependencias.
- Masivo: Listas y ``saveAll``.
- Constructores: Agilidad en tests.

